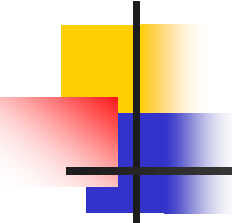


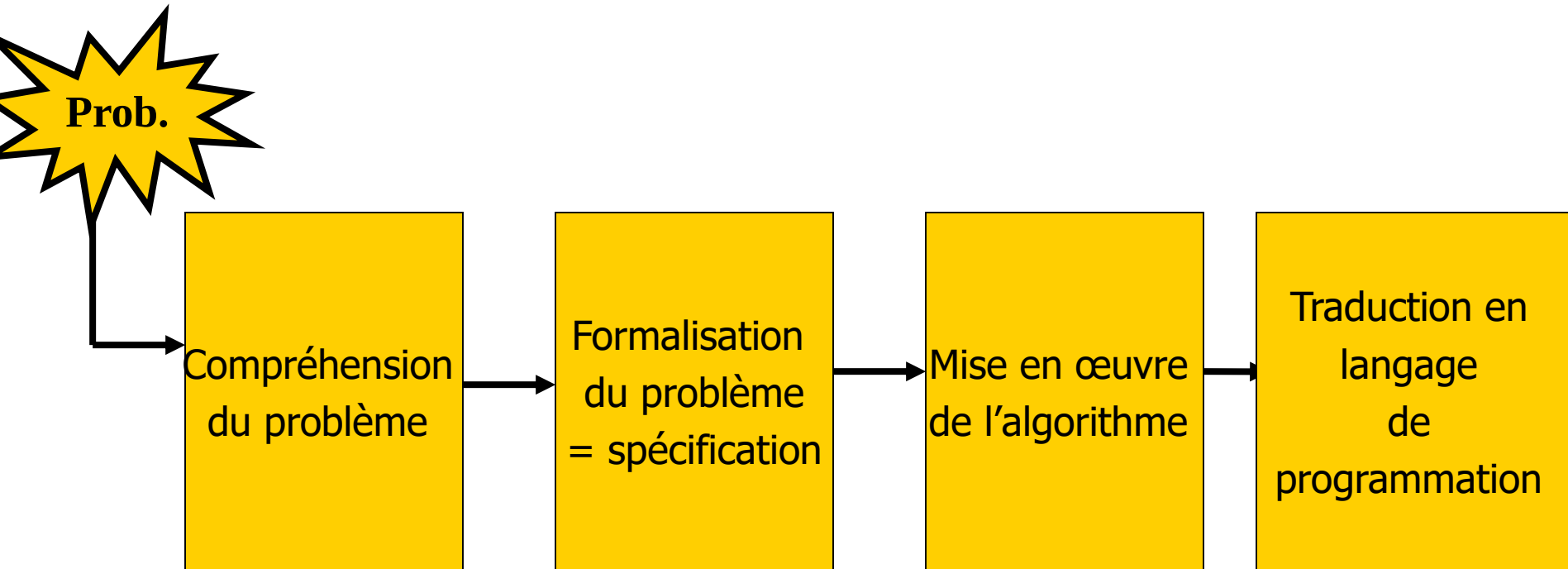
Introduction Générale & Concepts de Base

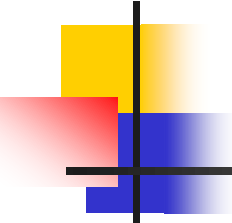


Plan

- TDA
- TD
- SD
- Pointeurs et références

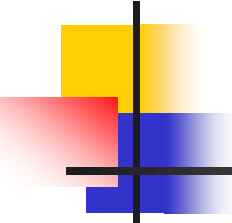
Démarche de résolution d'un problème





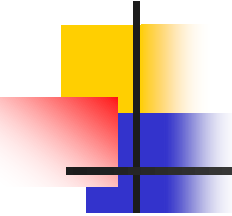
Rappel

- **Algorithme** : suite d'instructions qui s'exécute dans un ordre séquentiel et qui réalise un traitement
- **Programme** : algorithme traduit dans un langage de programmation (ADA, C++, Java, Cobol ...)



Types de Données Abstraits

- Chaque TDA est caractérisé par:
 - un nom
 - des opérations prévues sur le type.
 - La **description fonctionnelle** qui décrit les signatures des opérations en indiquant leurs noms et leurs ensembles de départ et d'arrivée
 - **description axiomatique** = sémantique
- Exemple:
 - entier : +, -, / , ...
 - liste : Vider une liste, Insérer un élément dans la liste, ...
- Un type de données abstrait est une **spécification** d'un TD



Exemple 1

nom
↙
type booléen

opérations

vrai : $\rightarrow$ booléen

faux : $\rightarrow$ booléen

$\neg$ _ : booléen $\rightarrow$ booléen

_ $\wedge$ _ : booléen $\times$ booléen $\rightarrow$ booléen

_ $\vee$ _ : booléen $\times$ booléen $\rightarrow$ booléen

signature ou syntaxe

sémantique

pour tout a, b : booléen

$\neg$ vrai = faux

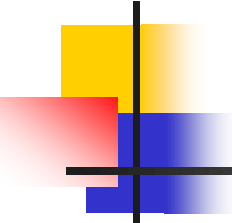
$\neg\neg a = a$

vrai $\wedge$ a = a

faux $\wedge$ a = faux

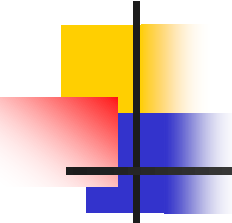
$a \vee b = \neg(\neg a \wedge \neg b)$

propriétés ou sémantique



Exemples 2

- **La description fonctionnelle pour TDA Ensemble = $\{\}$**
 - $\text{estVide} : \{\} \rightarrow \text{Booléen}$
 - $\text{appartient} : \text{Elément} \times \{\} \rightarrow \text{Booléen}$
 - $\text{ajouter} : \text{Elément} \times \{\} \rightarrow \{\}$
 - $\text{enlever} : \text{Elément} \times \{\} \rightarrow \{\}$
 - $\text{union} : \{\} \times \{\} \rightarrow \{\}$

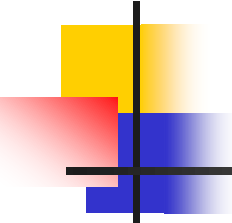


Exemples 3

■ La description axiomatique pour TDA $\mathbb{N}$

Axiomes de PEANO

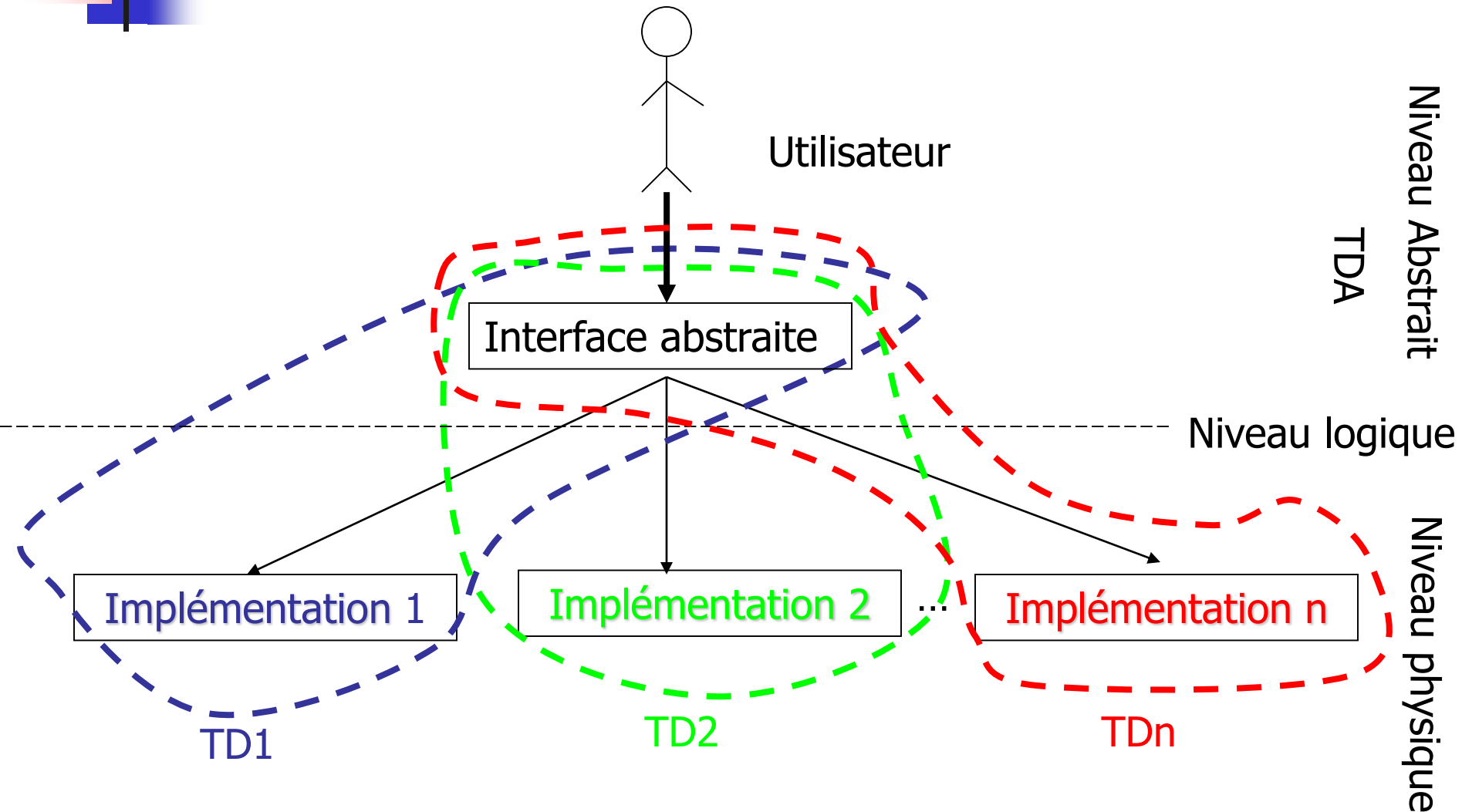
- $\forall n \in \mathbb{N} ; \exists n' / \text{succ}(n) = n'$
- $\forall n, n' \in \mathbb{N} ; n \neq n' \Rightarrow \text{succ}(n) \neq \text{succ}(n')$
- $\exists n \in \mathbb{N} / \text{succ}(n) = 0$
- $\forall n \in \mathbb{N} ; n + 0 = n$
- $\forall n, n' \in \mathbb{N} ; n + \text{succ}(n') = \text{succ}(n+n')$
- $\forall n \in \mathbb{N} ; n \times 0 = 0$
- $\forall n, n' \in \mathbb{N} ; n \times \text{succ}(n') = n + n \times n'$

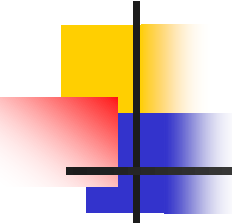


Types de Données

- TD **simple** ou élémentaire ou primitif (entier, booléen,...) : chaque variable de ce type peut prendre une seule valeur du domaine à la fois
- TD **composé** ou structuré (ou de référence en java) (tableau, enregistrement...): permet le stockage sous un même nom de variable de plusieurs valeurs de même type (exemple tableau d'entiers) ou non (exemple: type enregistrement)

Création d'un type de données





Implémentation d'un TDA

- Pour un même TDA plusieurs implémentations (=implantations) sont possibles
- Dans les langages OO le concept de classe facilite l'implémentation des TDA

Déclaration d'un TAD en C

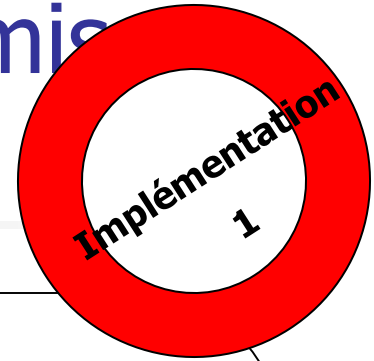
// fichier **pile.h**: déclaration du Type Abstrait de Données PILE
// Modèle de la structure de données :

```
typedef struct {  
    int size;  
    int* base;  
    int* top; } PILE;
```



// Fonctions et macros proposées :
void **init_pile**(PILE* p_pile, int siz);
int **push**(PILE* p_pile, int val);
int **pop**(PILE* p_pile, int* p_val);
void **del_pile**(PILE* p_pile);

Définition des opérations permises d'un TAD en C



// fichier pile.c

```
#include "pile.h"
```

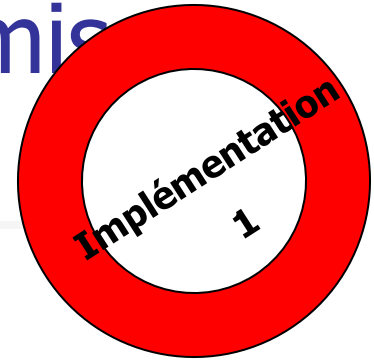
```
void init_pile(PILE* p_pile, int siz) { /* code de l'implémentation*/ }
```

```
int push(PILE* p_pile, int val) { /* code de l'implémentation*/ } }
```

```
int pop(PILE* p_pile, int* p_val) { /* code de l'implémentation*/ } }
```

```
void del_pile(PILE* p_pile) { /* code de l'implémentation*/ } }
```

Définition des opérations permises d'un TAD en C (plus précisément)



// fichier pile.c

```
#include "pile.h"
```

```
void init_pile(PILE* p_pile, int siz)
```

```
{ p_pile->size = siz;
```

```
p_pile->top = p_pile->base = malloc(siz * sizeof(int)); }
```

```
int push(PILE* p_pile, int val)
```

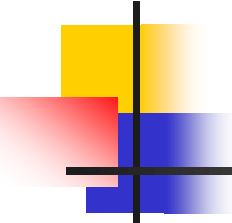
```
{ if ( p_pile->top == p_pile->base + p_pile->size ) return 0;
```

```
*p_pile->top++ = val; return 1; }
```

```
int pop(PILE* p_pile, int* p_val)
```

```
{ if ( p_pile->top == p_pile->base ) return 0; *p_val = *--p_pile->top; return 1; }
```

```
void del_pile(PILE* p_pile) { free(p_pile->base); p_pile->top = p_pile->base = 0;  
p_pile->size = 0; }
```



Utilisation d'un TDA

- Se fait **exclusivement** par l'intermédiaire des opérations qui leur sont associées et indépendamment de son implémentation
- En-têtes des fonctions et procédures du TDA : interface entre l'utilisateur et le TDA
 - Manipulation du TDA sans connaître son implémentation
 - Utilisation indépendante vis-à-vis de tout changement d'implantation

Utilisation d'un TDA en C

```
// Un programme d'essai
```

```
#include <stdio.h>
```

```
#include "pile.h"
```

```
main()
```

```
{
```

```
int i = 0;
```

```
PILE pile;
```

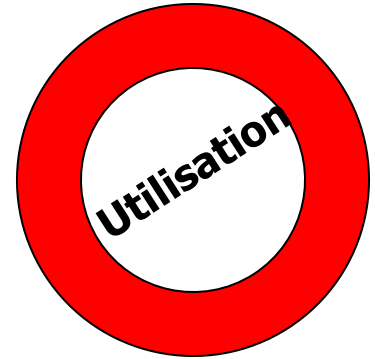
```
init_pile(&pile, 10);
```

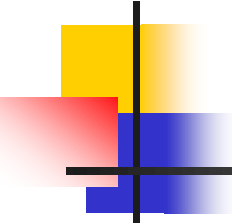
```
push(&pile, 20);
```

```
pop(&pile, &i)
```

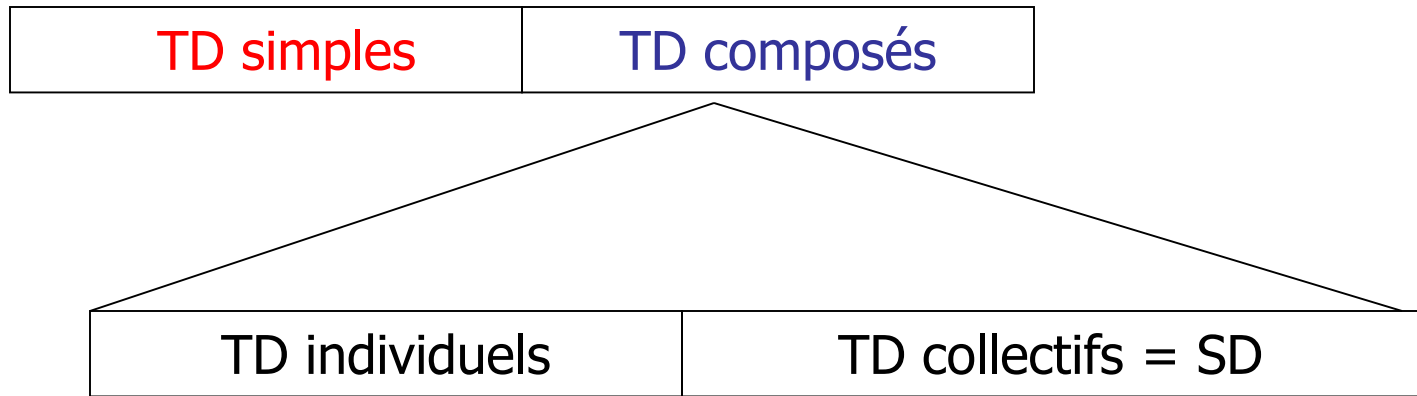
```
pile.size += 5;
```

```
del_pile(&pile);}
```



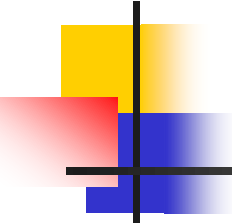


Structures de Données



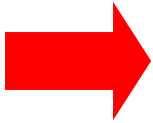
Exemple: Enregistrements

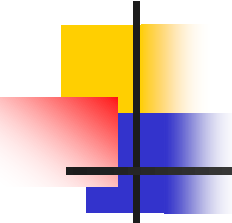
Exemple: liste, tableau, pile,
file, arbres ...



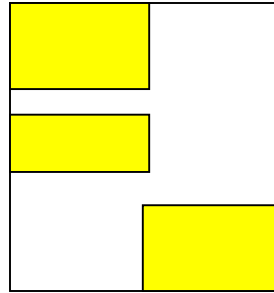
Structures de Données

- Les SD implémentées en utilisant des tableaux (contiguës) sont statiques → inconvénients :
 - Fixer le nombre maximal d'éléments dès le départ
 - Impossibilité d'allouer de l'espace mémoire non contigu même si cet espace est disponible mais fragmenté
 - Allocation d'un bloc MC avec une taille initiale fixe
 - Perte d'espace mémoire (espace occupé inutilement)





Solution : SD dynamiques

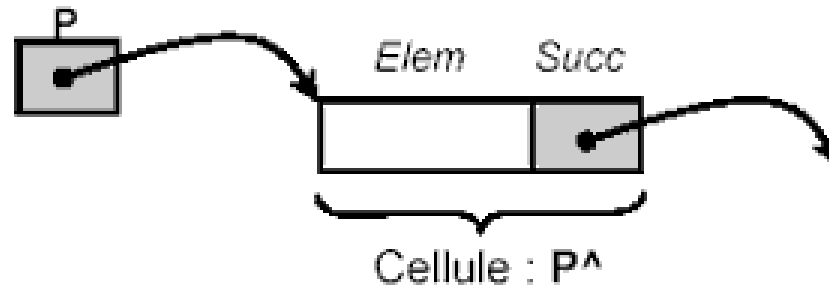


- Utilisation des ***pointeurs***

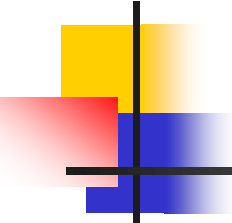
- Pas de contrainte de taille initiale
- Pas d'allocation inutile de l'espace mémoire
- Indépendance de la position des cases libres de la MC

Pointeurs

- Pointeur : cellule dont la valeur réfère à une autre cellule



- $\wedge$: symbole des pointeurs
- Deux procédures pour la gestion des pointeurs : Allouer et Libérer



Pointeurs

DefType

TypeCellule=...

DefVar

s (TypeCellule)

– variable statique de type TypeCellule

p (^Type Cellule)

– pointeur vers un objet de type TypeCellule

Début

...

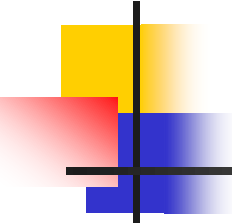
Allouer(p)

...

Libérer(p)

...

Fin



Pointeurs

Procédure **Allouer** (Res p: ^Type Cellule)

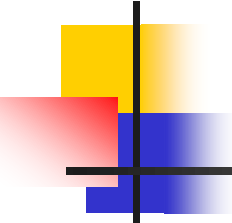
-- Précond:

-- Postcond: p contient l'adresse d'une nouvelle cellule
allouée

Procédure **Libérer** (Don p: ^Type Cellule)

-- Précond:

-- Postcond: libère la cellule d'adresse p



Rôle des pointeurs

- Supposons qu'on a 1000 étudiants et un traitement donné à faire (calcul moyenne, tri etc.):
 - 1) traitement statique : tableau de 1000 étudiants taille complète allouée dans la MC
 - 2) traitement dynamique:
 - on déclare le type étudiant
 - on déclare un pointeur sur le type étudiant= adresse qui peut contenir 1 étudiant
 - donc pour créer le premier étudiant il faut allouer un 1er espace puis un 2ième etc. Si on passe au 3ème étudiant et qu'on n'a plus besoin du premier on libère l'espace donc on alloue et on libère selon notre traitement